



École nationale
de la statistique
et de l'analyse
de l'information

MS Data Science pour la Connaissance Client

Apprentissage Statistique :

Prédiction du churn et de la valeur client dans le secteur des télécommunications

Réalisé par : FAGNINOUE Uriel et PRETET Baptiste

Année universitaire : 2025 – 2026

Table des matières

1	Introduction	4
2	Pré-traitement des données	4
2.1	Collecte des données	4
2.2	Traitement des valeurs manquantes	6
2.3	Statistiques descriptives	6
3	Clustering à titre exploratoire des données	9
4	Modélisation	11
4.1	Modélisation par régression	11
4.2	Modélisation par la classification	14
4.3	Combinaison de modèles : Voting et Stacking	15
5	Clustering (<i>Feature Engineering</i>) et transformation indicatrice Churn	19
5.1	Segmentation des profils de Churn par Clustering	20
5.2	Méthodologie et Validation	20
5.3	Analyse de la Stabilité	21
5.4	Utilisation pour la Classification	21
6	Classification multiple	22
7	Conclusion	28

Table des figures

1	Distribution des variables quantitatives	7
2	Matrice de corrélation	7
3	Boxplot de quelques variables par le Montant Total de charges	8
4	Dendrogramme et Tracé de la perte d'inertie	9
5	Profil client par cluster	10
6	Importance des variables	13
7	Répartition de la variable Churn	15
8	Comparaison des matrices de confusion : équilibre vs détection maximale	17
9	Top 15 des variables prédictives du Churn	18
10	Répartition du churn au cours des premiers mois	19
11	Distribution des différentes classes de Churn	20
12	Importance sans cluster	25
13	Importance avec cluster	25
14	Courbes Precision-Recall Multi-Classe (One-vs-Rest)	26
15	Matrice de confusion du modèle Random Forest	27

Liste des tableaux

1	Dictionnaire des variables	5
2	Statistiques descriptives des variables numériques	6
3	Performances des modèles avant et après optimisation	12
4	Synthèse des hyperparamètres testés et résultats de l'optimisation par GridSearch.	15
5	Synthèse des métriques de performance par modèle (triées par F1-score)	16
6	Métriques de performance après Clustering (triées par F1-score)	19
7	Comparaison de la distribution des clusters (Stabilité Train vs Test) . . .	21
8	Grille de recherche et configuration des modèles classification multiple .	23
9	Comparaison des modèles triés par F1-Score (Ordre Décroissant)	24

1 Introduction

Dans un contexte de forte concurrence, les entreprises de services font face à un enjeu central : fidéliser leurs clients tout en maximisant la valeur économique générée par chacun d’eux. La résiliation des contrats, communément appelée churn, constitue un coût important pour les entreprises, tant en raison de la perte de revenus futurs que des dépenses nécessaires à l’acquisition de nouveaux clients. Parallèlement, la capacité à estimer la valeur économique associée à un client est un levier essentiel pour orienter les stratégies marketing et commerciales.

L’objectif de ce projet est de mettre en œuvre une démarche complète de Machine Learning appliquée à une problématique de fidélisation client, en traitant simultanément un problème de classification et un problème de régression sur un même jeu de données. Le premier consiste à prédire la résiliation d’un client, tandis que le second vise à estimer sa valeur économique observée, mesurée par le montant total facturé.

2 Pré-traitement des données

2.1 Collecte des données

Pour commencer notre étude, nous avons constitué notre base de données à partir des informations clients d’une société de télécommunications. Le jeu de données utilisé est le **Telco Customer Churn Dataset**, disponible sur <https://www.kaggle.com/blastchar/telco-customer-churn>. Il contient les informations de 7043 clients et de 21 variables. Cette base de données permet d’analyser le comportement des clients, d’identifier les facteurs pouvant influencer la résiliation du service, et de prédire la valeur monétaire totale (TotalCharges) ainsi que le churn.

Nous pouvons lire ci-dessous le descriptif de la base :

Code	Type	Définition
gender	Catégorielle	Sexe du client (Male/Female)
SeniorCitizen	Binaire	Client senior (1 = oui, 0 = non)
Partner	Binaire	Client ayant un partenaire (Yes/No)
Dependents	Binaire	Client ayant des personnes à charge (Yes/No)
tenure	Numérique	Ancienneté du client (mois depuis l'abonnement)
PhoneService	Binaire	Service téléphonique actif (Yes/No)
MultipleLines	Catégorielle	Service de lignes multiples (Yes/No/No phone service)
InternetService	Catégorielle	Type de service Internet (DSL/Fiber optic/No)
OnlineSecurity	Catégorielle	Service de sécurité en ligne (Yes/No/No internet service)
OnlineBackup	Catégorielle	Service de backup en ligne (Yes/No/No internet service)
DeviceProtection	Catégorielle	Protection des appareils (Yes/No/No internet service)
TechSupport	Catégorielle	Support technique (Yes/No/No internet service)
StreamingTV	Catégorielle	Service de streaming TV (Yes/No/No internet service)
StreamingMovies	Catégorielle	Service de streaming films (Yes/No/No internet service)
Contract	Catégorielle	Type de contrat (Month-to-month/One year/Two year)
PaperlessBilling	Binaire	Facturation sans papier (Yes/No)
PaymentMethod	Catégorielle	Méthode de paiement (Bank transfer/Credit card/Electronic check/Mailed check)
MonthlyCharges	Numérique	Montant facturé mensuellement (USD)
TotalCharges	Numérique	Montant total facturé au client (USD)
Churn	Binaire	Résiliation du contrat (Yes/No)

TABLE 1 – Dictionnaire des variables

2.2 Traitement des valeurs manquantes

Les seules valeurs manquantes dans l'ensemble du jeu de données concernaient la variable `TotalCharges`, correspondant à des clients récemment abonnés n'ayant encore effectué aucun paiement. Nous avons choisi de remplacer ces valeurs par 0 afin de conserver ces clients dans l'analyse. Aucune autre variable ne présentait de valeurs manquantes.

2.3 Statistiques descriptives

Pour en apprendre davantage sur notre variable d'intérêt et nos variables explicatives, nous allons procéder à une analyse statistique descriptive.

Statistique	tenure	MonthlyCharges	TotalCharges
Count	7043	7043	7043
Mean	32.37	64.76	2279.73
Std	24.56	30.09	2266.79
Min	0.00	18.25	0.00
25%	9.00	35.50	398.55
50%	29.00	70.35	1394.55
75%	55.00	89.85	3786.60
Max	72.00	118.75	8684.80

TABLE 2 – Statistiques descriptives des variables numériques

Les statistiques descriptives fournies donnent un aperçu complet des caractéristiques clés des données étudiées. Avec un échantillon de 7043 observations, ces statistiques couvrent une gamme variée de variables qui représentent différents aspects des clients de la télécommunication. Cependant, il est important de noter que ces statistiques ne fournissent qu'une vue d'ensemble. Par exemple, le montant facturé au client présente une grande variabilité, allant de 0 \$ à 8684 \$, avec une moyenne de 2279.73 \$ ce qui reflète à la fois la présence de clients récents n'ayant encore rien payé et de clients fidèles ayant accumulé des charges importantes. Tout ceci est confirmé par la distribution très asymétriques à droite des charges totales sur la figure 1 ci-dessous

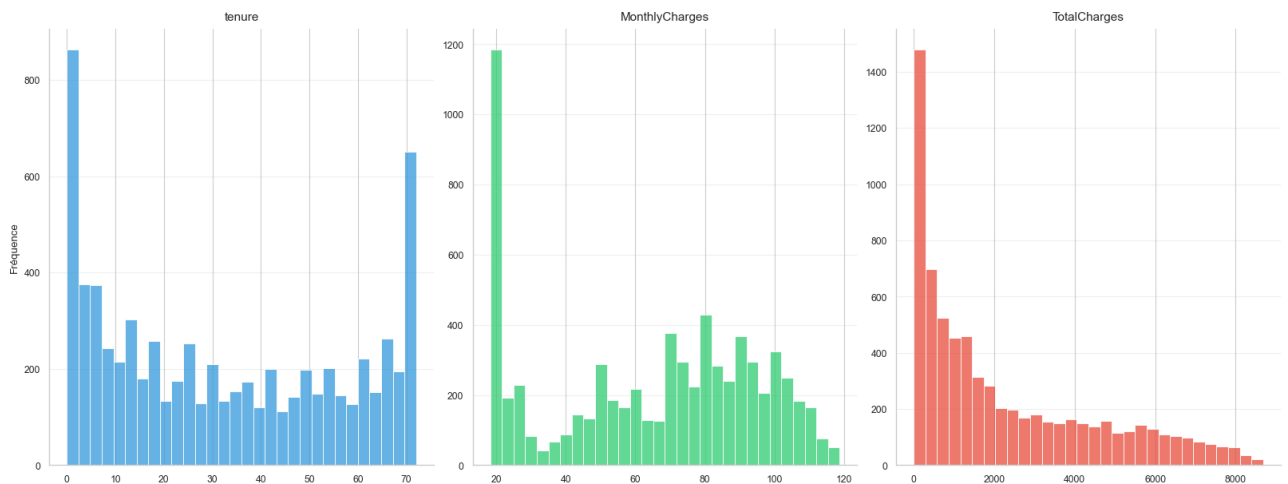


FIGURE 1 – Distribution des variables quantitatives

Afin d’explorer les relations entre deux variables, nous entreprenons une analyse de la corrélation. Cette approche nous permet d’évaluer la force et la direction de l’association statistique entre les variables.

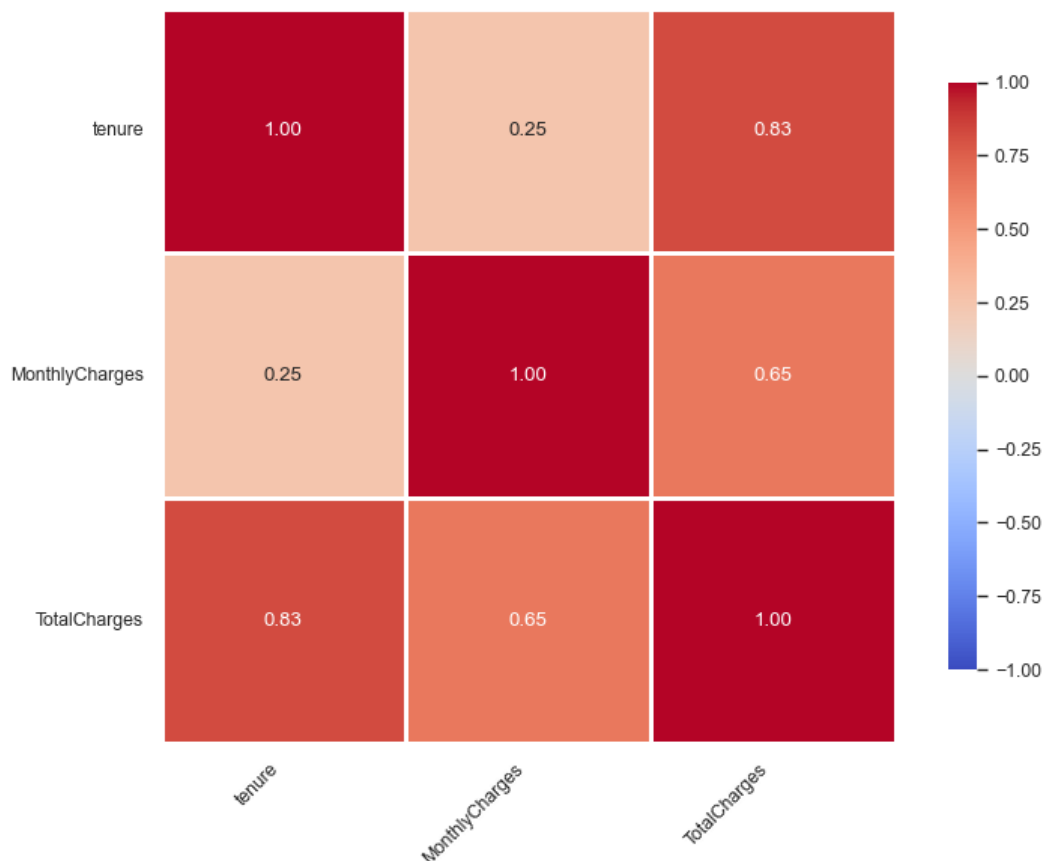


FIGURE 2 – Matrice de corrélation

La matrice de corrélation (figure 2) montre une forte corrélation positive entre tenure et TotalCharges (0,83), indiquant que le montant total facturé augmente avec l’ancienneté

du client. On observe également une corrélation modérée entre MonthlyCharges et TotalCharges (0,65), suggérant que les clients ayant des forfaits plus coûteux accumulent plus de charges dans le temps. En revanche, la corrélation entre tenure et MonthlyCharges est faible (0,25), ce qui indique que le niveau de facturation mensuelle est relativement indépendant de la durée d'abonnement.

Analyse sur quelques variables explicatives

Pour mieux comprendre la relation entre nos deux variables dépendantes et nos variables explicatives, nous recourons à la représentation graphique des variables qualitatives. Cela nous permet d'explorer visuellement les liens entre ces différentes variables.

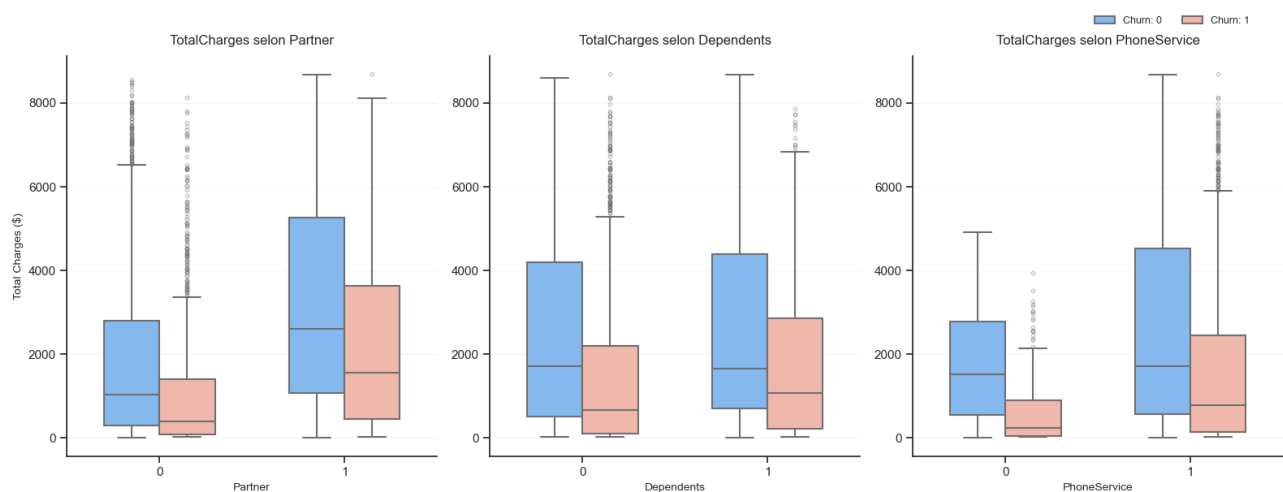


FIGURE 3 – Boxplot de quelques variables par le Montant Total de charges

L'analyse des TotalCharges selon les variables Partner, Dependents et PhoneService, en distinguant les clients ayant churné de ceux restés fidèles, met en évidence des différences marquées de comportement. Dans l'ensemble, les clients n'ayant pas résilié leur contrat présentent des TotalCharges significativement plus élevées, traduisant une ancienneté plus importante. À l'inverse, les clients ayant churné affichent systématiquement des montants cumulés plus faibles, quelle que soit la catégorie considérée. A titre d'exemple, les clients disposant d'un partenaire ou de personnes à charge présentent des TotalCharges plus élevées, suggérant une plus grande stabilité et un engagement plus durable envers le service.

3 Clustering à titre exploratoire des données

Nous avons décidé d'explorer de manière plus approfondie la structure sous-jacente de nos données, identifier des regroupement de profils de clients homogène. Pour cela, nous mettons en œuvre une approche de Clustering non supervisée, en combinant plusieurs méthodes dans le but d'obtenir une segmentation robuste.

Pour obtenir une classification robuste, il est possible de combiner la méthode de la Classification Ascendante Hiérarchique (CAH) avec celle des k-means. La CAH permet dans un premier temps de déterminer le nombre optimal de classes k , qui est ensuite utilisé comme paramètre d'initialisation pour la méthode des k-means. On obtient le résultat suivant grâce à la CAH

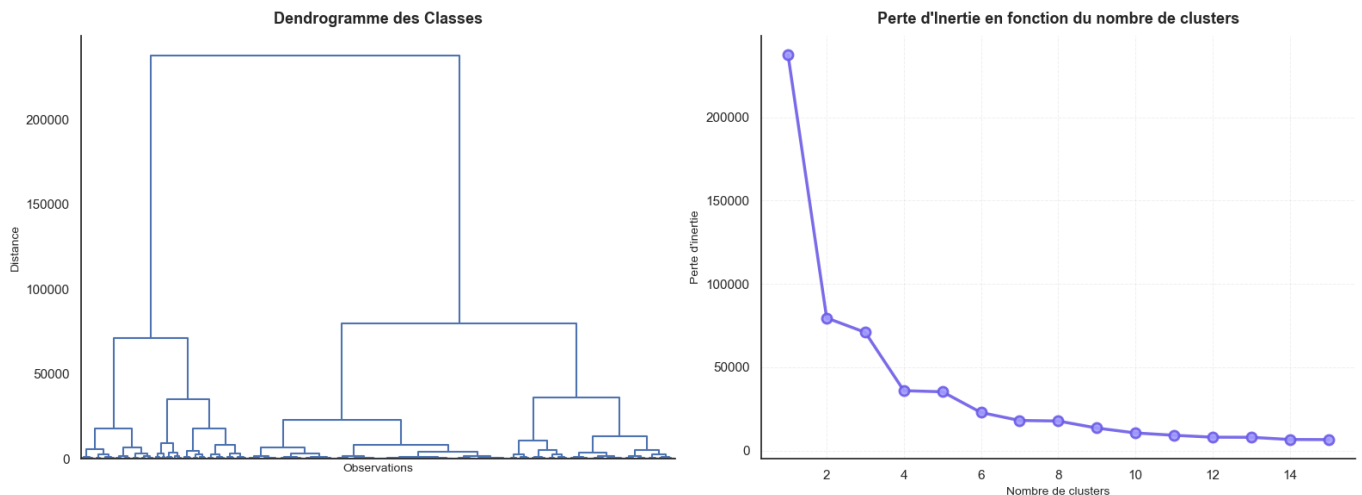


FIGURE 4 – Dendrogramme et Tracé de la perte d'inertie

Le tracé de la perte d'inertie nous incite à choisir une partition en 4 groupes (lecture de gauche à droite : juste avant le coude ou changement de pente s'opérant au passage de 3 à 4 groupes). Pour initialiser l'algorithme kmeans nous utilisons les résultats trouvés par la CAH : le nombre de classe optimal $k = 4$ de la CAH. En utilisant ce nombre de classes optimal ($k = 4$), nous appliquons ensuite l'algorithme des k-means. La projection des clusters en deux dimensions est réalisée à l'aide de l'algorithme t-SNE, permettant une visualisation synthétique des profils clients.

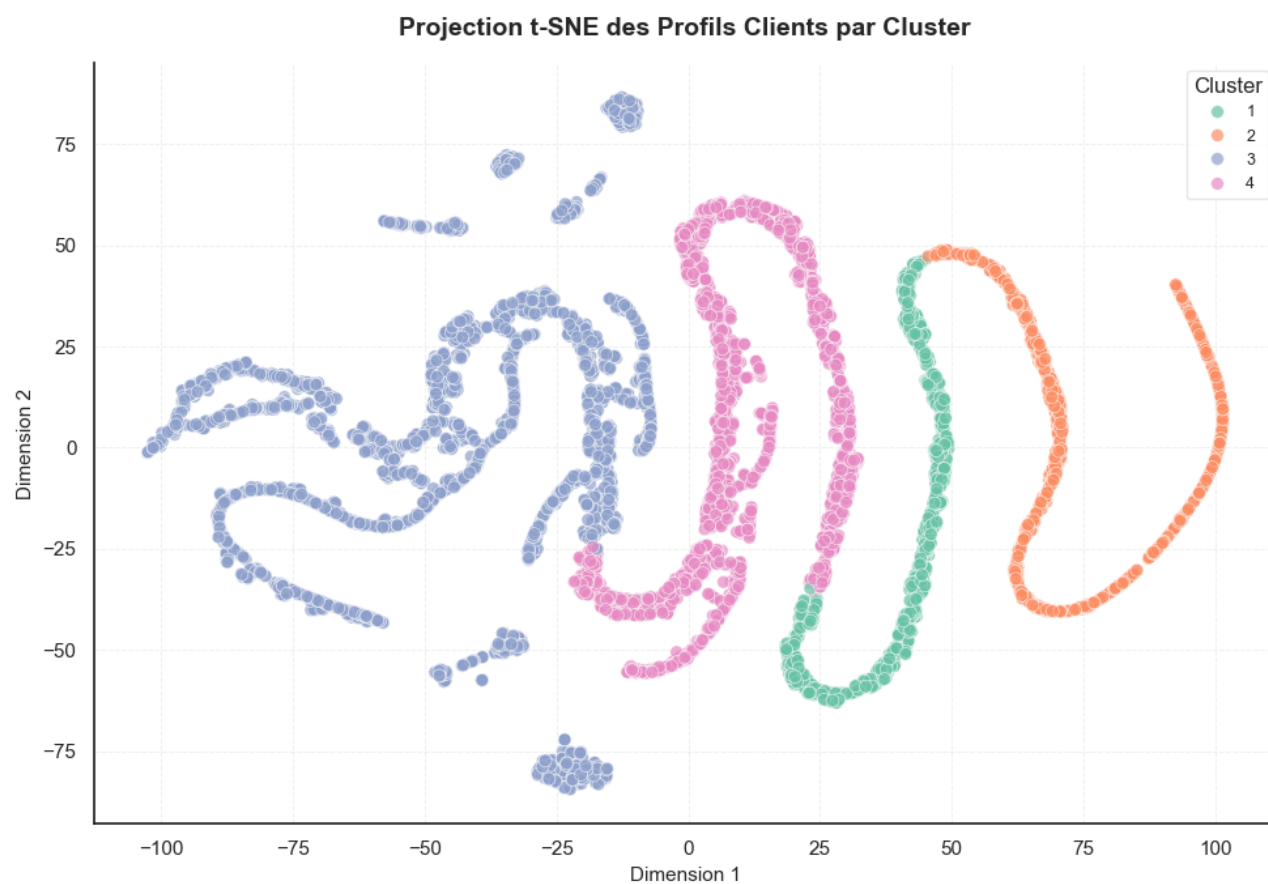


FIGURE 5 – Profil client par cluster

Nous avons évalué la qualité de nos clusters avec l'indice Silhouette. Nous avons obtenu **0,583** ce qui indique une bonne segmentation malgré la forme du nuage de points des clusters.

4 Modélisation

Après avoir identifié des profils clients homogènes à travers l'analyse de Clustering, permettant une meilleure compréhension de la structure du jeu de données, nous adoptons une approche de machine learning supervisé afin d'aller au-delà de cette analyse descriptive. Deux problématiques complémentaires sont alors étudiées : d'une part un problème de régression visant à prédire le montant total facturé au client (`TotalCharges`), et d'autre part, un problème de classification consistant à prédire la résiliation du contrat (`Churn`).

4.1 Modélisation par régression

La première problématique abordée concerne la prédiction du montant total facturé au client (`TotalCharges`), variable continue reflétant la valeur économique générée par un client sur la durée. Ce problème est traité à l'aide de méthodes de régression supervisée

Méthodologie commune aux modèles de régression

Afin d'assurer une comparaison cohérente entre les différentes méthodes de régression, une méthodologie identique est appliquée à l'ensemble des modèles étudiés.

Le jeu de données est tout d'abord scindé en deux sous-ensembles : un échantillon d'entraînement représentant 80% des observations et un échantillon de test représentant 20% des observations. Cette séparation permet d'évaluer la capacité prédictive des modèles sur des données non observées lors de l'apprentissage. Pour chaque méthode, trois étapes successives sont mises en œuvre :

- une première estimation à l'aide d'un modèle de base, utilisant des paramètres par défaut ;
- une recherche hyperparamètres via validation croisée afin d'améliorer les performances prédictives ;
- l'évaluation du modèle optimisé sur l'échantillon de test.

Par ailleurs, il est important de préciser que nous incluons pas la variable `Churn` dans le modèle de régression car elle correspond à une information de nature postérieure au processus de facturation, son inclusion introduirait ainsi un biais d'information.

Les performances des modèles sont évaluées à l'aide de deux métriques d'erreur : MAE (Mean Absolute Error) et la RMSE (Root Mean Squared Error). Ces indicateurs permettent une interprétation directe des erreurs de prédiction en unités monétaires, tout en pénalisant différemment les erreurs importantes.

Synthèse comparative des différents résultats obtenus

Nous présentons à présent les résultats des performances des modèles avant et après optimisation évaluées sur le jeu test à l'aide des différents métriques RMSE et MAE

Modèle	Optimisation	RMSE	MAE
Knn	Non	461.51	306.83
Knn	Oui	382.31	257.64
Random Forest	Non	80.80	53.35
Random Forest	Oui	80.08	52.922
XGBoost	Non	86.59	57.39
XGBoost	Oui	78.85	51.847

TABLE 3 – Performances des modèles avant et après optimisation

Le tableau 3 présente les performances des différents modèles de régression évalués, avant et après optimisation des hyperparamètres, en termes de RMSE et de MAE. On constate dans un premier temps que l'optimisation des hyperparamètres du modèle KNN a permis une amélioration des erreurs. On note que le RMSE et qui passe respectivement de 461.51 à 382.31 et la MAE qui passe de 306.83 à 257.64.

Ensuite, pour le modèle Random Forest, l'optimisation n'apporte qu'une amélioration mineure : le RMSE passe de 80.80 à 80.08 et la MAE de 53.35 à 52.92. Cette évolution très faible montre que le modèle Random Forest est déjà performant avec ses hyperparamètres par défaut et que l'optimisation n'apporte qu'un gain limité. Cela reflète également la grande flexibilité du modèle, qui peut être sensible au sur-apprentissage ou au bruit, même après réglage des paramètres.

Pour le modèle XGBoost l'optimisation des hyperparamètres a permis une baisse simultanée du RMSE et de la MAE. Cela témoigne de la robustesse de ce modèle face au sur-apprentissage, grâce à sa régularisation intégrée et à sa capacité à gérer efficacement

les interactions entre variables. Ainsi, le modèle XGBoost optimisé apparaît ainsi comme le plus performant et le plus stable parmi les méthodes étudiées.

On s'intéresse par la suite à l'importance des variables dans le modèle XGBOOST retenu.

Analyse de l'importance des variables

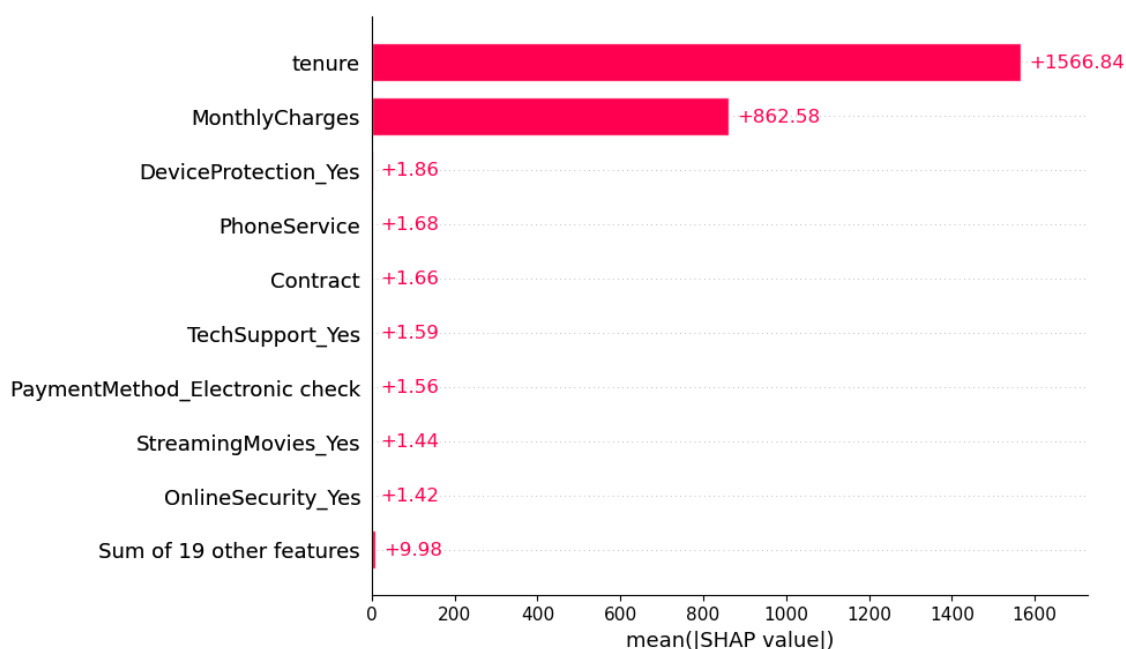


FIGURE 6 – Importance des variables

Le graphique de la figure 6 révèle la contribution de chaque variable aux modèles de prédictions du modèle XGBoost. On note que l'ancienneté du client (Tenure) est la variable la plus influente du modèle. Plus précisément plus un client est fidèle plus il accumulera des charges totales. S'en suit en deuxième position les charges mensuelles avec un impact positif significatif montrant que des charges mensuelles élevées contribuent à augmenter les prédictions de TotalCharges. Le reste des variables présente des contributions très faible.

En conclusion, notre modèle repose essentiellement sur deux variables principales, cette simplicité du modèle constitue un atout en termes d'interprétabilité et de robustesse, tout en maintenant d'excellentes performances (RMSE=78, MAE=51). Toutefois, cette forte dépendance à l'ancienneté et aux charges mensuelles représente également une limite : la prédiction de TotalCharges devient moins pertinente pour les clients récents,

notamment ceux dont l'ancienneté est très faible ou nulle, pour lesquels l'information historique est insuffisante.

4.2 Modélisation par la classification

Dans cette partie, nous allons tenter de modéliser la prédiction de l'attrition (*churn*) de nos clients (soit le fait de résilier son abonnement). Pour ce faire, nous évaluerons différentes méthodologies et modèles, ainsi que leur impact sur la précision de ces prédictions.

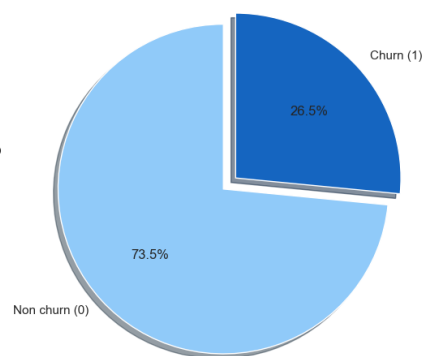
Classification binaire

Dans un premier temps, nous chercherons simplement à prédire si un client va se désengager ou rester fidèle. Afin d'y parvenir, nous proposons d'évaluer plusieurs modèles de classification classiques et d'analyser si leur combinaison (*voting* ou *stacking*) ou l'usage de méthodes de *boosting* permet d'améliorer leurs performances. Comme nous l'avons vu dans la première partie, les variables `TotalCharges` et `MonthlyCharges` plus `tenure` sont corrélées. Nous enlèverons donc `TotalCharges` des variables.

Nous utiliserons également l'algorithme `GridSearchCV` afin d'optimiser la recherche de nos hyperparamètres.

Optimisation par GridSearch : Le *Grid Search* est une méthode d'optimisation qui consiste à définir un ensemble fini de valeurs pour chaque hyperparamètre d'un modèle et à tester l'intégralité des combinaisons possibles. La performance de chaque configuration est évaluée par validation croisée ($cv=5$). Dans le cadre de notre étude, nous avons choisi le **F1-score** comme métrique de référence. Ce choix se justifie par la nécessité de trouver un équilibre entre la précision et le rappels.

En complément de ces hyperparamètres, nous avons appliqué l'argument `class_weight='balanced'`. Car comme le montre cette figure, notre jeu de données présente un déséquilibre de classes, avec environ 80 % de clients fidèles contre seulement 20 % de clients en situation d'attrition (*churners*).



L'utilisation de ce paramètre permet donc de pénaliser davantage les erreurs de classification sur la classe minoritaire, évitant ainsi que le modèle ne développe un biais systématique en faveur de la classe majoritaire.

FIGURE 7 – Répartition de la variable Churn

Modèle	Grille de recherche (Params)	Meilleurs paramètres
KNN	<code>n_neighbors</code> : [3, ..., 20], <code>weights</code> : ['unif.', 'dist.']}	<code>n_neighbors</code> : 17, <code>weights</code> : 'uniform'
Decision Tree	<code>max_depth</code> : [None, 5, 10, 15, 20]	<code>max_depth</code> : 5
Logistic Reg.	<code>C</code> : [0.1, 1, 10]	<code>C</code> : 1
Random Forest	<code>n_estimators</code> : [50, 100, 150], <code>max_depth</code> : [...]	<code>max_depth</code> : 10, <code>n_estimators</code> : 150
SVM	<code>C</code> : [0.1, 1, 10], <code>kernel</code> : ['rbf']	<code>C</code> : 1, <code>kernel</code> : 'rbf'
LightGBM	<code>n_est.</code> : [100, ..., 250], <code>learn_rate</code> : [0.005, ..., 0.1]	<code>learn_rate</code> : 0.01, <code>n_est.</code> : 150
XGBoost	<code>n_est.</code> : [100, 150, 200], <code>max_depth</code> : [3, 4, 5, 6]	<code>max_depth</code> : 3, <code>n_est.</code> : 100
Gaussian NB	Paramètres par défaut	{}
AdaBoost	<code>n_est.</code> : [50, 100], <code>learn_rate</code> : [0.01, 0.1, 1.0]	<code>learn_rate</code> : 1.0, <code>n_est.</code> : 100

TABLE 4 – Synthèse des hyperparamètres testés et résultats de l'optimisation par Grid-Search.

4.3 Combinaison de modèles : Voting et Stacking

Afin d'augmenter nos performances, nous avons implémenté deux approches d'agrégation. L'objectif est de réduire la variance et de compenser les erreurs individuelles des modèles.

Voting Classifier : Nous avons utilisé un *Soft Voting* basé sur un trio robuste : **Random Forest**, **SVM** et **Régression Logistique**. Cette méthode pondère les décisions en fonction des probabilités de chaque modèle.

Stratégies de Stacking : Le *Stacking* utilise un méta-modèle (Régression Logistique) pour apprendre à combiner les prédictions. Nous avons testé trois configurations :

- **Performance** : Basée sur l'excellence du Boosting (*XGBoost*, *LightGBM*).
- **Diversité** : Mélange de familles d'algorithmes (*XGBoost*, *Random Forest*, *KNN*) pour capturer des patterns variés.
- **Stable** : Une combinaison équilibrée pour assurer la régularité des prédictions.

Le tableau suivant présente les résultats obtenus pour l'ensemble des modèles testés, classés par leur score F1.

Modèle	Accuracy	Precision	Recall	F1-score	ROC-AUC
LightGBM	0.767	0.544	0.759	0.634	0.842
Decision Tree	0.757	0.528	0.791	0.633	0.837
Voting	0.769	0.551	0.709	0.620	0.839
Random Forest	0.765	0.543	0.719	0.619	0.837
SVM	0.744	0.511	0.778	0.617	0.819
Logistic Regression	0.741	0.508	0.775	0.614	0.838
XGBoost	0.720	0.484	0.826	0.611	0.835
Stacking_perf	0.796	0.628	0.570	0.597	0.842
Stacking_div	0.796	0.627	0.567	0.596	0.838
Stacking	0.794	0.624	0.564	0.593	0.841

TABLE 5 – Synthèse des métriques de performance par modèle (triées par F1-score)

L'examen de ses métriques révèle que nos modèles rencontrent des difficultés avec la **Précision**, traduisant une tendance à générer des faux positifs. Cependant, l'analyse détaillée permet de distinguer des comportements variés :

- **Meilleur compromis (F1-Score)** : Le **LightGBM** (0,634) obtient le meilleur score, suivi de très près par l'Arbre de Décision.
- **Détection maximale (Recall)** : Si l'objectif est de ne rater aucun départ, le **XG-Boost** est le plus performant avec un rappel de **0,826**.
- **Précision (Precision)** : Les modèles de **Stacking** sont supérieurs pour limiter les faux positifs ($\approx 0,62$), mais au détriment du Rappel.
- **Capacité de séparation (ROC-AUC)** : L'ensemble de nos modèles affiche de bons scores (0,842 pour LightGBM), confirmant leur capacité à distinguer les deux classes.

Interprétation par les matrices de confusion : En comparant les matrices de la figure 8, on observe des stratégies divergentes. Le **LightGBM** présente un profil plus équilibré, tandis que le **XGBoost** maximise la couverture des *churners* (Recall élevé) au prix d'un nombre plus important de faux positifs.

Dans notre stratégie d'entreprise, nous privilégions le **Recall** à la Précision : il est commercialement préférable de solliciter par erreur un client fidèle plutôt que de manquer un départ effectif, ce qui représenterait une perte de revenus nette.

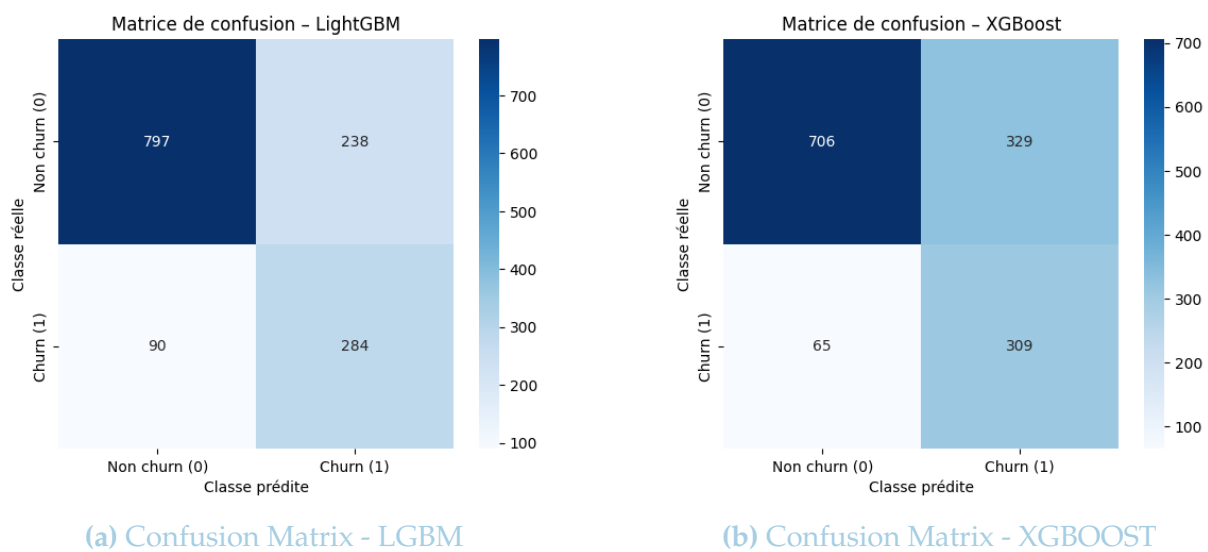


FIGURE 8 – Comparaison des matrices de confusion : équilibre vs détection maximale

En ce qui concerne l'importance de nos variables (Figure 9), on peut voir que *MonthlyCharges*, *tenure*, et *Contract* sont les plus significatives. Cette hiérarchie révèle plusieurs points clés :

- **Aspect financier :** Les frais mensuels (*MonthlyCharges*) indiquant que le coût du service est le premier levier de désengagement.
- **Fidélité et engagement :** La variable *tenure* (ancienneté) montre que le risque de churn évolue fortement avec le temps passé chez l'opérateur. De même, le type de *Contract* souligne que les clients engagés sur le long terme sont plus stables que ceux ayant des contrats mensuels.
- **Services techniques :** L'importance de variables comme *PaymentMethod_Electronic check* ou *InternetService_Fiber optic* suggère que certains modes de paiement ou types de technologies influencent également la satisfaction client.

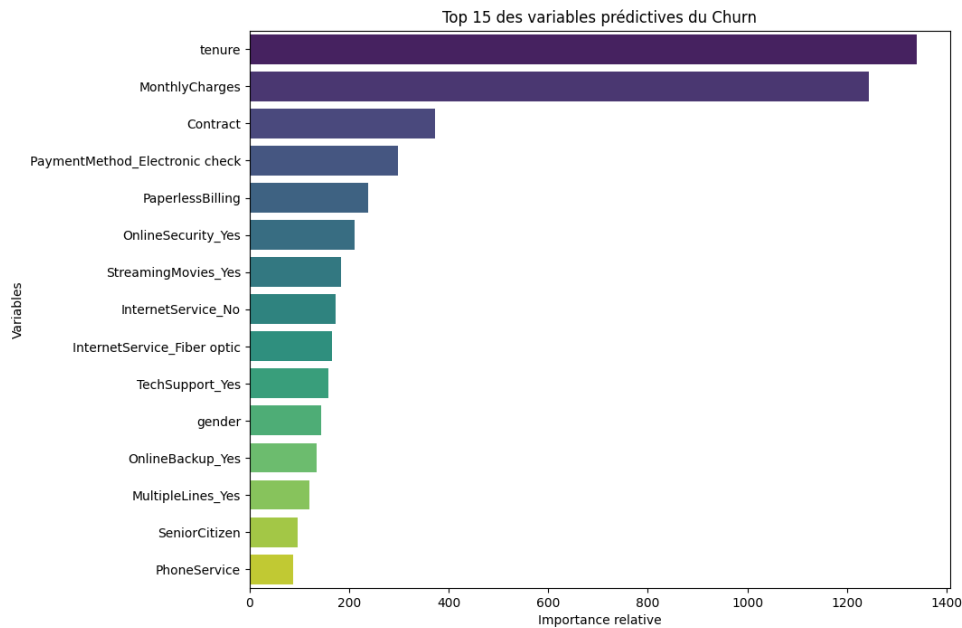


FIGURE 9 – Top 15 des variables prédictives du Churn

Bien que nos modèles, notamment l'*Arbre de Décision* et le *LightGBM*, affichent des capacités de distinction satisfaisantes, la stagnation de la précision suggère que les variables actuelles ne permettent pas de capturer toute la complexité des comportements de désengagement. Afin d'améliorer ces performances, nous proposons d'explorer l'**ingénierie des variables** (*Feature Engineering*). Cette étape permettrait de créer des indicateurs plus **performants**, notamment via la génération de **clusters de clients**. L'objectif est d'utiliser un modèle *K-means* pour segmenter la base de données et ainsi détecter des structures comportementales latentes qui n'ont pas été identifiées par nos modèles de classification actuels. De plus, si l'on s'intéresse à une application métier de cette classification, il est bien plus intéressant de connaître la période à laquelle le client va *churner* plutôt que de simplement savoir s'il va *churner* ou non. Nous allons donc nous attacher à modéliser ce phénomène afin d'avoir un impact métier plus concret. Nous conserverons les méthodologies précédemment utilisées. Le but sera de garder ou d'améliorer les performances que nous avons atteintes en classification binaire.

Cependant le tableau suivant présente les performances des modèles après l'intégration des nouveaux indicateurs issus du clustering. On peut voir que certains modèles se sont **améliorés**. Le modèle le plus stable reste tout de même le *LightGBM*. On peut donc conclure que le clustering a bien un impact ; nous développerons la méthodologie ainsi que son impact sur la **classification multiple** dans les prochaines parties.

Modèle	Accuracy	Precision	Recall	F1-score	ROC-AUC
LightGBM	0.774	0.556	0.735	0.633	0.836
Random Forest	0.766	0.544	0.727	0.622	0.837
Voting	0.774	0.559	0.701	0.622	0.840
SVM	0.748	0.517	0.775	0.620	0.819
XGBoost	0.730	0.495	0.824	0.618	0.833
Logistic Regression	0.747	0.515	0.767	0.617	0.841
Decision Tree	0.743	0.511	0.770	0.614	0.831
Stacking_div	0.796	0.627	0.575	0.600	0.837
Stacking	0.800	0.640	0.561	0.598	0.843
Stacking_perf	0.789	0.606	0.583	0.594	0.839

TABLE 6 – Métriques de performance après Clustering (triées par F1-score)

5 Clustering (*Feature Engineering*) et transformation indicatrice Churn

Dans cette partie, nous allons étudier le délai nécessaire avant qu'un client ne mette fin à son contrat. Pour ce faire, nous analysons la *tenure* (ancienneté) des clients en corrélation avec leur statut de résiliation. Comme l'illustre la figure 10, on observe qu'une majorité de clients partent durant les trois premiers mois. Au-delà de cette période, le volume de *churn* diminue de manière significative.

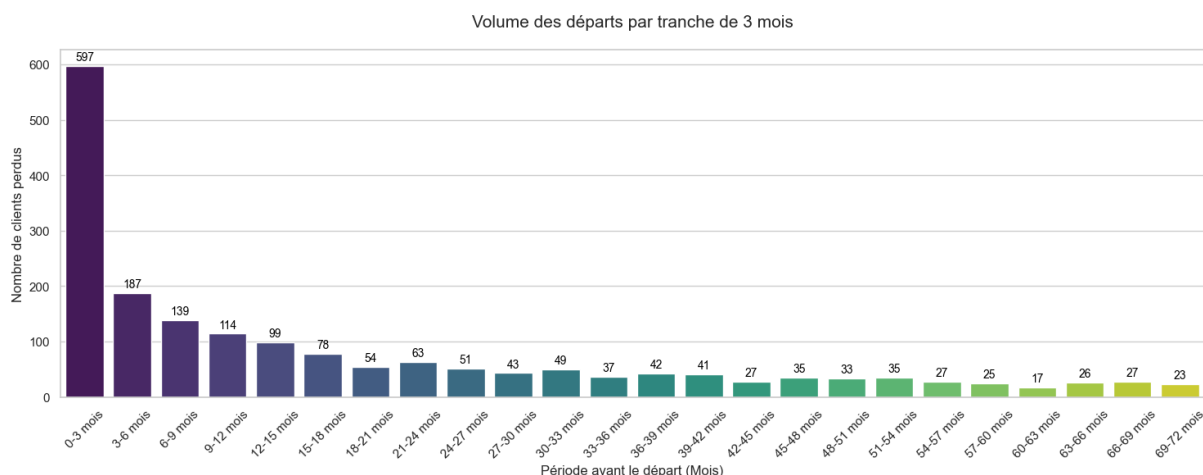


FIGURE 10 – Répartition du churn au cours des premiers mois

Afin d'optimiser l'apprentissage de notre modèle, nous avons défini des classes de prédiction équilibrées au niveau du Churn. Cette approche permet de répartir les différents profils de *churners* de manière homogène entre les catégories, favorisant ainsi

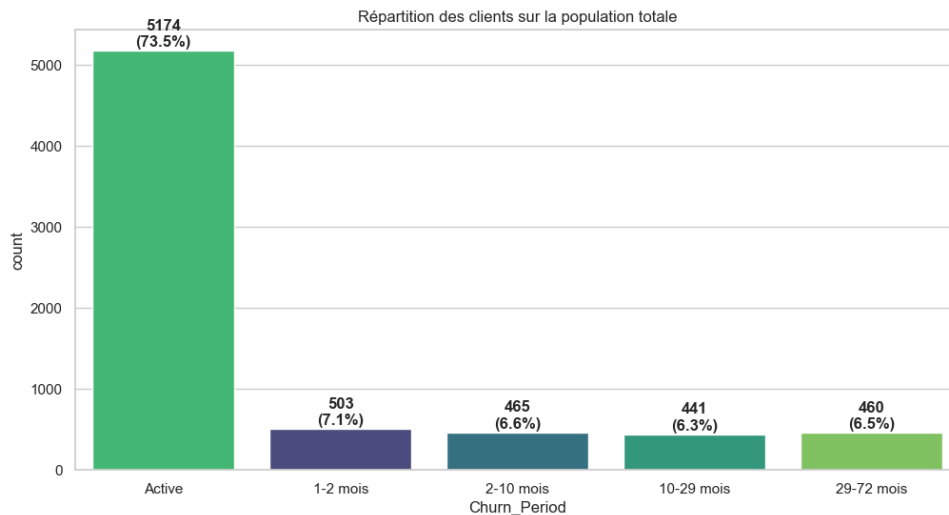


FIGURE 11 – Distribution des différentes classes de Churn

la performance des prédictions futures. Tout comme pour la classification binaire, nous observons initialement un déséquilibre entre les groupes de *churners* et de *non-churners*. Nous corrigerons ce biais via l'utilisation du paramètre `class_weight='balanced'` dans nos modèles.

5.1 Segmentation des profils de Churn par Clustering

Une fois nos différentes classes de *churn* établies, nous procédons, comme dans la première partie, à un *clustering* sur ces variables à l'aide de l'algorithme des K-means. En suivant la même méthodologie, nous avons utilisé un dendrogramme ainsi que la méthode du coude (*elbow method*) pour identifier le nombre optimal de clusters. Pour ce jeu de données, le choix s'est porté sur 8 clusters.

5.2 Méthodologie et Validation

Afin d'évaluer la robustesse de notre segmentation, nous avons scindé nos données en ensembles d'entraînement (*train*) et de test. Le *clustering* est entraîné sur le *train*, puis sa stabilité est vérifiée par projection sur le *test*.

Les indicateurs de performance obtenus sont les suivants :

- **Coefficient de Silhouette (0,346)** : Ce score supérieur à 0,25 indique une structure de groupe raisonnable et une cohésion interne satisfaisante pour ce type de

données comportementales.

- **Indice de Davies-Bouldin (1,234)** : Ce résultat témoigne d’une séparation correcte entre les clusters (plus l’indice est faible, meilleure est la distinction).
- **Indice Calinski-Harabasz (2358,5)** : Ce score élevé confirme que les clusters sont globalement bien définis par rapport à la dispersion totale.

5.3 Analyse de la Stabilité

Le tableau suivant présente la répartition des observations au sein des clusters pour les deux ensembles :

Cluster	Train %	Test %
0	8,25	8,16
1	10,37	10,65
2	21,58	22,00
3	11,70	10,65
4	11,50	12,28
5	12,21	11,00
6	9,92	9,87
7	14,47	15,40

TABLE 7 – Comparaison de la distribution des clusters (Stabilité Train vs Test)

Interprétation : Comme le montrent ces résultats, les écarts de proportion entre le *train* et le *test* sont minimes (généralement inférieurs à 1 %). Cette convergence démontre que le modèle de *clustering* est stable et généralise bien les comportements des clients, sans phénomène de sur-apprentissage.

5.4 Utilisation pour la Classification

Une fois ces segments validés, nous enregistrons le modèle de *clustering*. Ces clusters seront intégrés comme de nouvelles variables explicatives (*features*) lors de la phase de classification. Cette approche hybride permet d’enrichir le modèle prédictif en lui fournissant des informations structurelles sur les profils de consommation, facilitant ainsi la détection du *churn*.

6 Classification multiple

Une fois l'étape de *feature engineering* terminée, nous allons l'utiliser pour prédire le délai avant qu'un client ne risque de partir (*churn*). Pour ce faire, nous reprenons la séparations en différentes classes de Churn effectuée précédemment. Dans cette partie nous garderons TotalCharges pour retirer tenure et MonthlyCharges.

Pour la séparation des données (*split*) en un dataset de training et de testing, nous séparons nos nouveaux labels. De plus, nous soumettons nos ensembles X_{train} et X_{test} à notre modèle de clustering afin de prédire l'appartenance de chaque observation à un groupe spécifique.

Une fois le jeu de données prêt, nous réalisons une classification en deux temps :

1. Sans la variable de cluster.
2. Avec l'intégration de la variable de cluster.

L'objectif de cette classification multiple est de vérifier s'il est possible d'obtenir des prédictions aussi précises que nos modèle binaire, tout en récupérant des informations métier plus riche que précédement. Nous pourrons également mesurer l'impact de l'ajout des clusters sur la performance globale. Nous avons choisi de réutiliser les modèles précédents en ajustant leurs hyperparamètres pour cette tâche spécifique. Pour les modèles sensibles au déséquilibre des classes (comme XGBoost ou SVM), nous utilisons l'argument (*class weights*).

TABLE 8 – Grille de recherche et configuration des modèles classification multiple

Modèle	Grille de recherche	Configuration fixe (v2)
KNN	n_neighbors : [10, 20, 30]	weights : 'distance'
Decision Tree	depth : [8, 12, 20], leaf : [10, 20]	class_weight : 'balanced'
Logistic Reg.	C : [0.01, 0.1, 1]	multinomial, balanced
Random Forest	depth : [10, 15], 20, split : [5, 10]	n_est : 200, balanced, max_samples : 0.8
SVM	C : [0.1, 1], kernel : ['rbf']	class_weight : 'balanced'
LightGBM	n_est : [100, 150, 200], lr : [0.1, 0.01, 0.05]	leaves : [31, 50], balanced
XGBoost	n_est : [100, 150], lr : [0.01, 0.05]	depth : [3, 4, 5], sample_weight (y)
Gaussian NB	-	Paramètres par défaut
Stacking	-	Final : LogReg (C=0.1, balanced)

Une fois les paramètres du modèle choisis nous analysons les performances de ses modèles. Le tableau précédent compare les résultats obtenus sans la présence de la variable cluster et avec sa présence .

Analyse Comparative des Performances des Modeles

Le tableau suivant compare les résultats obtenus sans l'ajout de clusters et avec l'ajout de clusters. Les meilleures performances pour chaque modèle sont indiquées en **gras**.

TABLE 9 – Comparaison des modèles triés par F1-Score (Ordre Décroissant)

Modèle	Clusters	Accuracy	Precision-W	Recall-W	F1-Weighted	ROC-AUC-W
Random Forest	Non	0.7757	0.7696	0.7757	0.7706	0.8594
	Oui	0.7722	0.7685	0.7722	0.7684	0.8584
Voting	Oui	0.7764	0.7587	0.7764	0.7637	0.8636
	Non	0.7715	0.7565	0.7715	0.7611	0.8638
LightGBM	Oui	0.7615	0.7660	0.7615	0.7629	0.8300
	Non	0.7509	0.7625	0.7509	0.7554	0.8352
Decision Tree	Non	0.7026	0.7252	0.7026	0.7119	0.6806
	Oui	0.6962	0.7215	0.6962	0.7069	0.6783
KNN	Non	0.7537	0.7154	0.7537	0.7119	0.8054
	Oui	0.7360	0.6933	0.7360	0.7047	0.7771
Stacking	Oui	0.6778	0.8132	0.6778	0.7032	0.8642
	Non	0.6742	0.8103	0.6742	0.7000	0.8631
XGBoost	Non	0.6118	0.8145	0.6118	0.6357	0.8700
	Oui	0.6026	0.8147	0.6026	0.6269	0.8713
Logistic Reg.	Oui	0.6068	0.7994	0.6068	0.6334	0.8550
	Non	0.5997	0.8027	0.5997	0.6297	0.8646
SVM	Oui	0.5990	0.7835	0.5990	0.6326	0.8481
	Non	0.5976	0.7857	0.5976	0.6305	0.8487
Gaussian NB	Non	0.4194	0.7837	0.4194	0.4554	0.8019
	Oui	0.4053	0.7767	0.4053	0.4321	0.8373

Observations principales

- **Stabilité Accuracy vs Rappel** : On remarque une identité entre l'Accuracy et le Recall-W pour tous les modèles.
- **Meilleure performance globale** : Le **Random Forest** sans clustering domine le classement avec le meilleur F1-Score (0.7706).
- **Impact positif du Clustering** : Le clustering améliore les performances de LightGBM, Voting et Stacking, mais dégrade celles de KNN et du Decision Tree.
- **Impact négatif du Clustering** : Le clustering nuit légèrement au Voting Classifier et dégrade les performances de **XGBoost** de façon marquée (baisse du F1 de 0.6538 à 0.6230).
- **Précision élevée** : XGBoost et Stacking conservent la meilleure Precision-W (> 0.81), indiquant une forte fiabilité sur les prédictions positives malgré un rappel plus modeste.

On observe que l'ajout du clustering en tant que simple variable n'a pas eu d'impact majeur sur notre modélisation. En revanche, la segmentation du jeu de données en

différentes classes (basée sur les périodes de churn) a eu un effet significatif. En effet, on note une augmentation drastique de la précision sur l'ensemble des modèles.

Pour la classification binaire, la précision oscillait auparavant entre **0,495** et **0,640** ; elle se situe désormais entre **0,710** et **0,815**, ce qui représente une nette amélioration. Nos modèles semblent donc mieux minimiser les faux positifs, ce qui entraîne mécaniquement une progression du F1-score.

Pour classer nos churneurs, nous choisirons le modèle **Random Forest**. Bien que ses résultats soient proches du *Voting Classifier* en termes de rappel (*recall*), il est plus simple à mettre en œuvre et moins gourmand en ressources.

En comparant l'importance des variables (*feature importance*) des modèles avec et sans clusters, on remarque que la variable cluster elle-même a un faible impact. Nous retrouvons le même duo que pour la classification binaire : **Contract** et **TotalCharges**. Il est toutefois intéressant de noter que **TotalCharges** semble jouer un rôle bien plus important pour prédire nos différentes classes de churn.

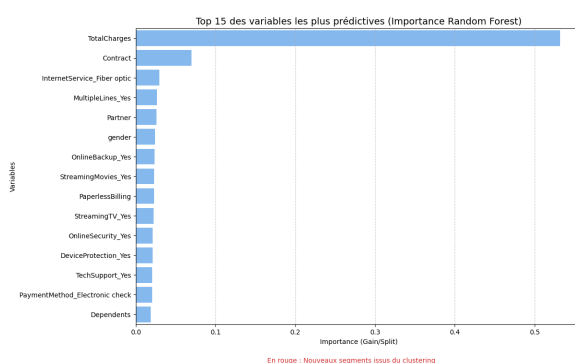


FIGURE 12 – Importance sans cluster

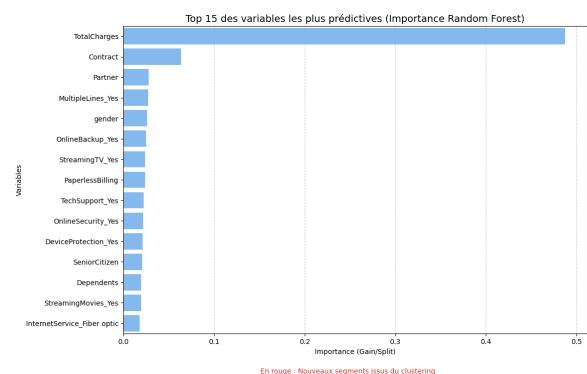


FIGURE 13 – Importance avec cluster

Pour une évaluation plus fine de notre modèle, nous allons analyser la matrice de confusion ainsi que la courbe Précision-Recall par classe. Cette approche nous permet de dissocier la performance globale de la capacité de détection sur chaque segment de clientèle.

Analyse des courbes Précision-Recall par classe

L'analyse de la courbe Précision-Recall (PR) révèle une performance globale solide avec un *Average Précision* (AP) global de 0,86. Cependant, on observe une hétérogénéité marquée selon l'ancienneté des clients (tenure) :

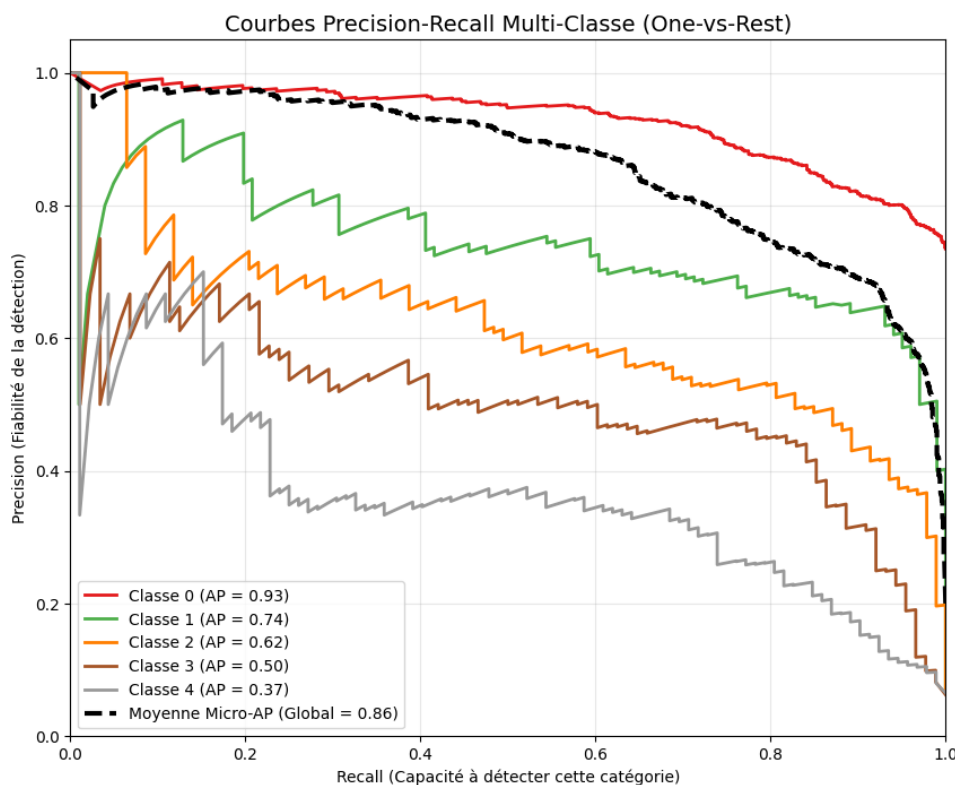


FIGURE 14 – Courbes Precision-Recall Multi-Classe (One-vs-Rest)

- **Classe 0 (Non-churneurs - AP : 0,93)** : La performance est excellente. Le modèle identifie presque parfaitement les clients fidèles, avec une courbe restant très proche du coin supérieur droit.
- **Classe 1 (Churn 1-2 mois - AP : 0,74)** : Cette classe, cruciale pour notre stratégie de rétention immédiate, affiche une performance solide. C'est un point positif car elle représente notre priorité opérationnelle.
- **Classes 2 & 3 (Churn 2-29 mois - AP : 0,62 & 0,50)** : Les performances sont moyennes avec des oscillations. Cela indique que pour maintenir un rappel élevé (détecter tous les churneurs), la précision chute rapidement.
- **Classe 4 (Churn 29-72 mois - AP : 0,37)** : Il s'agit du point faible du modèle. Les clients à très longue ancienneté sont les plus difficiles à prédire, générant de nombreux faux positifs dès que l'on cherche l'exhaustivité.

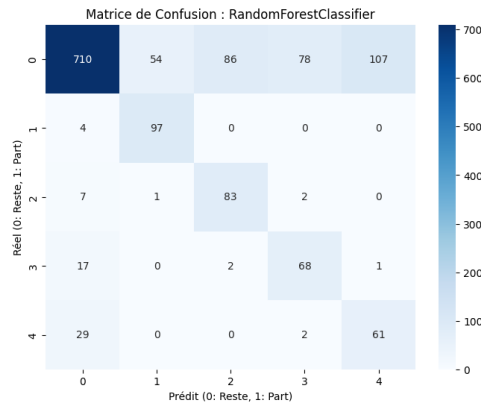


FIGURE 15 – Matrice de confusion du modèle Random Forest

D'un point de vue métier, cette dégradation de la performance avec l'ancienneté est à nuancer : la Classe 1 est notre priorité absolue. Plus l'échéance de churn est lointaine (Classes 3 et 4), moins l'impact immédiat sur nos campagnes de rétention est critique.

On observe que la classe 0 semble être sur-prédite, ce qui s'explique par son poids prédominant dans l'échantillon (déséquilibre de données). Par conséquent, la performance globale du modèle est tirée vers le haut par cette classe majoritaire, masquant une dégradation de la précision sur les autres classes. Plus on descend dans les classes (de 1 à 4), plus le taux de faux négatifs (le modèle manque la vraie classe) devient préoccupant, car le modèle a tendance à classer ces individus par erreur dans la classe 0. Classe 1 : C'est la classe la mieux prédite après la 0 (97 bons résultats, presque aucun faux négatif vers les classes 2, 3, 4). Elle semble avoir des caractéristiques très distinctes.

Enfin, il est important de noter que ces résultats sont obtenus sur un dataset restreint de seulement 7 000 entrées avec un nombre limité de variables. On peut légitimement s'attendre à une amélioration significative de la précision et de la robustesse des modèles en enrichissant le jeu de données avec davantage de variables comportementales et un historique plus vaste.

7 Conclusion

Ce projet a permis de concevoir une démarche complète de Machine Learning appliquée à la connaissance client. En traitant simultanément la valeur économique et le risque de résiliation, nous avons tenté de répondre à des enjeux de fidélisation et de rentabilité.

Synthèse des résultats

- **Estimation de la valeur client (Régression) :** Le modèle XGBoost s'est imposé comme le plus robuste après optimisation avec une RMSE de 78,85 et une MAE de 51,85. L'analyse a démontré que la valeur d'un client repose quasi exclusivement sur son ancienneté (*tenure*) et ses charges mensuelles, offrant un modèle simple à interpréter.
- **Prédiction du Churn (Classification) :**
 - En classification binaire, le LightGBM a offert le meilleur compromis avec un F1-score de 0,634, bien que le XGBoost soit préférable pour une détection maximale des départs avec un *Recall* de 0,826 mais avait trop de faux négatif.
 - Le passage à une classification multiple (prédiction de la période de départ) a radicalement amélioré la précision de nos modèles, atteignant des scores de précision compris entre 0,710 et 0,815. Nous avons décidé de retenir le Random Forest en raison de ses performances et surtout sa complexité faible.
- **Apport du Clustering :** L'utilisation du clustering pour l'ingénierie des variables (*feature engineering*) n'a pas eu d'impact majeur sur la performance globale, hormis une légère amélioration pour certains modèles comme le LightGBM.

Perspectives

L'étude souligne que la prédiction du churn de client sur le long terme restent plus complexes (Classe 3 et 4). Afin d'améliorer les résultats obtenus, un enrichissement du jeu de données avec des variables plus complètes (historique de navigation, interactions avec le support technique, etc.) ainsi qu'un volume d'observations plus important serait idéal pour renforcer la robustesse des prédictions.